Critical Thinking

1. **Explain the difference between a has-a and is-a relationship among the classes.**

- Has-a: Occurs when one class uses an instance of another class as a member field.
- Is-a: The relationship demonstrated by a class derived from an existing class.

2. **If a base class has a public method go() and a derived class has a public method stop(), which methods will be available to an object of the derived class?**

- Both methods go() and stop() will be available to an object of the derived class, as all the methods from the parent class will be inherited to the subclass and the subclass can take access to methods within itself and also with its parent class.

3. **Compare and contrast implementing an abstract method to overriding a method.**

- An abstract method is a method that has been declared, but not implemented. Abstract methods appear in abstract classes. They are implemented in a subclass that inherits the abstract class.
- So basically, an abstract method is a method that has been declared but empty. When implementing that method to the subclass, the subclass will be able to re-design that method to work in the way the subclass or that method meant to work.

4. **Compare and contrast an abstract class to an interface.**

- An abstract method is a method that has been declared, but not implemented. Abstract methods appear in abstract classes. They are implemented in a subclass that inherits the abstract class.
- Interface is a class with abstract methods. An interface cannot be inherited, but it can be implemented by any number of classes.
- Same: Contains empty methods and allows subclasses to implement those classes.
- Difference:
    - Abstract: Subclass can only extend with one abstract class, but allows subclass to extend methods that are needed.
    - Interface: Subclass can implement multiple interfaces. However, if the subclass already implements the interface, it has to implement every method inside that interface.

5. **Use the following classes to answer the questions below:**

```
interface Wo {
        public int doThat();
}

public class Bo {
        private int x;
        public Bo(int z) {
                x = z;
        }

        public int doThis() {
                return(2);
        }
```

```
        public int doNot() {
                return(15);
        }
}

public class Roo extends Bo implements Wo {
        public Roo {
                super(1);
        }

        public int doThis() [
                return(10);
        }

        private int doThat() {
                return (20);
        }
}
```

a) What type of method is doThat() in Wo?

- doThat() is called an abstract method, as it has been declared without a body and inside an interface.

b) What is Wo?

- Wo is an interface. Because it is created with an abstract method.

c) Why is doThat() implemented in Roo?

- Because the Roo class already implements Wo, so that Roo must implement all the abstract methods from Wo.

d) List the methods available to a Roo object.

- doThat()
- doThis()
- doNot()

e) How does the implementation of doThis() in Roo affect the implementation of doThis() in Bo?

- When the code is run, the computer will prioritize the value doThis() in Roo more than in Bo.

f) What action does the statement super(1) in Roo perform?

- Because in Bo, the constructor is assigned with a variable so that as a subclass, Roo has to call for that variable [public Bo(int x)]

g) Can the doThis() method in Bo be called from a Roo object? If so, how?

- No, because Java will always run the overridden method in the sub classes. In this class, Roo already updated the value of doThis() so the method in Bo cannot be run.

h) Can a method in Roo call the doThis() method in Bo? If so, how?

- Yes. A method inside Roo can call Bo's version specifically using the super method. This only works from within Roo's own method not from outside